The DeepSeek Attention Lineage

MLA → NSA → DSA → CSA

One campaign on the KV-cache / attention bottleneck, told in four moves

June 2026

Four DeepSeek attention variants are usually discussed as separate tricks. They are better read as **one continuous campaign against the same bottleneck**: the attention layer costs $O(L^2)$ compute and keeps an $O(L)$ key–value (KV) cache, and at long context that cache — not the weights — dominates inference memory and bandwidth. Each method pulls one or more of *three levers*.

Lever 1 — Per-token compression	**Lever 2 — Sequence compression**	**Lever 3 — Selection / sparsity**
shrink each token's cached vector	merge positions into fewer KV entries	attend to fewer positions

MLA pulls lever 1. NSA pulls levers 2+3 (coarse). DSA combines lever 1 (inherited) with lever 3 (fine). CSA composes all three. That single sentence is the whole report; the rest makes it precise.

Notation. Throughout, $h_t \in \mathbb{R}^d$ is the input hidden state at position t; q, k, v are query/key/value vectors; there are n_h heads of width d_h; and $\mathrm{Attn}(q, K, V) = \mathrm{softmax}\big(qK^\top/\sqrt{d_k}\big)\,V$ denotes ordinary scaled dot-product attention over whatever (K, V) a method exposes. Every method below differs only in *how it builds* (K, V) — and in *what it must cache*.

What "compression" means here (read once — it applies to every method below).

Every method compresses or drops things on the *key/value* side — the tokens attended *to*. The *query* side is never compressed: there is always one query per position, so **each layer always outputs L vectors**. Compression just makes the attention map *tall and thin*, $L \times M$ with $M < L$, instead of $L \times L$. So **nothing is ever "expanded back" to full length** — there is no downsample→upsample as in a U-Net, and the compressed KV is consumed inside the softmax, never reconstructed.

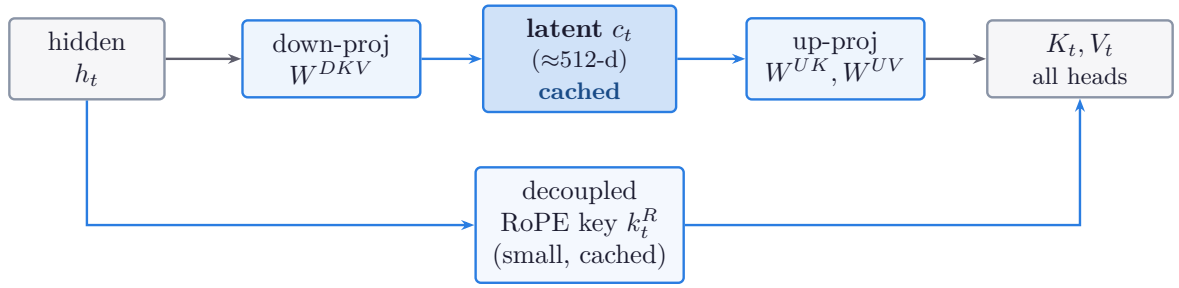
Two flavours to keep separate: **MLA** shrinks *each* KV vector but keeps all L of them (*per-token* compression); **NSA's compression branch** and **CSA / HCA** pool *several positions into one* (*sequence-length* compression). Selection methods (**DSA**, **Stem**) don't pool at all — they *drop* tokens, again only on the KV side.

1 MLA — Multi-head Latent Attention

Lever 1: per-token compression | dense over all tokens

Problem. Standard multi-head attention (MHA) caches a full key and value *per head, per token*: memory $\propto L \cdot n_h \cdot d_h$. With many heads and a long context this cache is the binding constraint.

Idea. Don't cache K and V. Instead project the hidden state down to a single small *latent* vector c_t (e.g. 512-dim) and cache *that*. At attention time, every head's K and V are reconstructed by up-projection from the shared latent. Because the low-rank compression is incompatible with rotary position encoding, MLA *decouples* a small RoPE-carrying sub-vector that is kept separately.



The math. With down/up projections W^{DKV}, W^{UK}, W^{UV} and a decoupled RoPE key, MLA caches a single low-rank latent instead of per-head K, V:

$$\underbrace{c_t^{KV} = W^{DKV} h_t}_{\textbf{cached},\ \in \mathbb{R}^{d_c}}, \qquad k_{t,i}^C = W_i^{UK} c_t^{KV}, \quad v_{t,i}^C = W_i^{UV} c_t^{KV}, \qquad \underbrace{k_t^R = \mathrm{RoPE}(W^{KR} h_t)}_{\textbf{cached},\ \text{shared over heads}}.$$

Each head's key is a content part concatenated with the shared RoPE part; the query is built the same way from a separately compressed query latent c_t^Q:

$$q_{t,i} = [\, q_{t,i}^C ;\, q_{t,i}^R \,], \qquad k_{t,i} = [\, k_{t,i}^C ;\, k_t^R \,], \qquad o_{t,i} = \mathrm{Attn}(q_{t,i}, K_{\le t,i}, V_{\le t,i}^C), \quad u_t = W^O [o_{t,1}; \ldots; o_{t,n_h}].$$

It is also cheap at *inference* because the content score factorises — W^{UK} absorbs into the query projection, so attention runs *directly against the cached latent* without ever forming k^C:

$$q_{t,i}^{C\top} k_{j,i}^C = c_t^{Q\top} (W_i^{UQ\top} W_i^{UK}) c_j^{KV}.$$

Effect. KV cache shrinks to one latent (+ tiny RoPE part) per token instead of $2 n_h d_h$ — roughly a **14×** reduction in DeepSeek-V2 — with quality *matching or beating* MHA, because the up-projection is learned and absorbed into the query/output projections at inference.

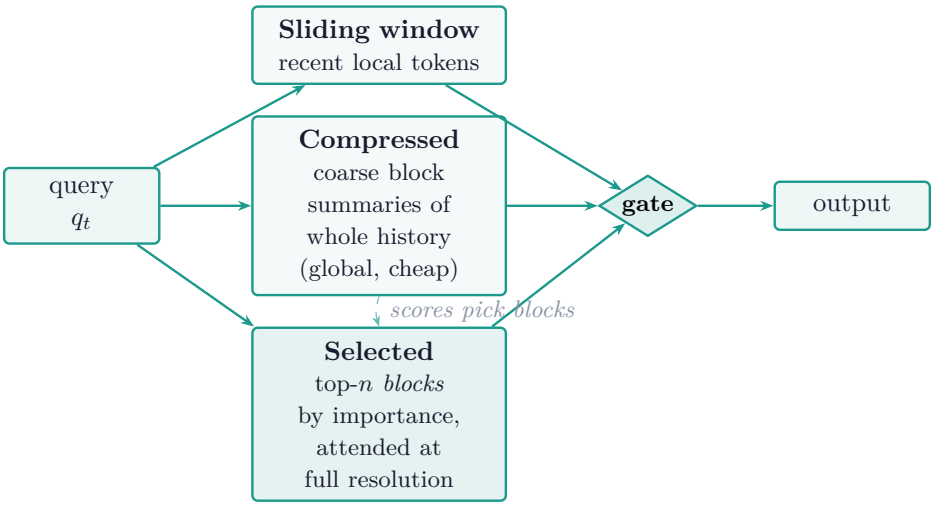
What MLA does *not* do. It still attends *densely to every past token*. It makes each token cheaper to store; it does not reduce *how many* tokens you look at. That is the next lever.

2 NSA — Native Sparse Attention

Lever 3: selection | Lever 2: coarse compression | trained from scratch

Problem. Sparsity that is bolted on *only at inference* degrades quality and maps poorly to GPUs. Full attention is still $O(L^2)$. NSA's thesis: make sparsity **native** — trainable end-to-end — and **hardware-aligned** — block-structured so memory access stays coalesced.

Idea. For each query, run *three parallel branches* and combine them with a learned gate:



The **compressed** branch gives a cheap global view (a form of sequence compression). The **selected** branch reuses the compressed scores to choose the top-n blocks, then attends to them at full resolution — recovering precision. **Block** granularity (not per-token) is the key hardware choice: it keeps reads contiguous, so the custom Triton kernels stay arithmetic-bound.

The math. Let $k_{1:t}, v_{1:t}$ be the per-head keys/values; the three branches build three different (K, V) sets. *(i) Compression* — a learnable φ (MLP + intra-block position encoding) maps each length-l block (stride d) to one token:

$$\tilde{k}_i^{cmp} = \varphi(k_{id+1:\,id+l}), \quad \tilde{v}_i^{cmp} = \varphi(v_{id+1:\,id+l}), \qquad o_t^{cmp} = \mathrm{Attn}(q_t, \tilde{K}_t^{cmp}, \tilde{V}_t^{cmp}).$$

The compressor is a *trained* map, not an average: with learnable intra-block position vectors $p_1, \ldots, p_l$ and $[\cdot]$ concatenating the l in-block vectors, $\varphi(x_{1:l}) = \mathrm{MLP}_\theta([\,x_1 + p_1;\ \ldots;\ x_l + p_l\,])$. *(ii) Selection* — reuse the compression scores $p_t^{cmp} = \mathrm{softmax}(q_t^\top \tilde{K}_t^{cmp})$ as block importance, remap onto selection blocks of size l' to get p_t^{slc}, keep the top-n blocks, and attend at full resolution:

$$\mathcal{I}_t = \text{top-}n(p_t^{slc}), \quad K_t^{slc} = [\,k_{jl'+1:\,jl'+l'} : j \in \mathcal{I}_t\,], \quad o_t^{slc} = \mathrm{Attn}(q_t, K_t^{slc}, V_t^{slc}).$$

(iii) Sliding window — the last w tokens, $o_t^{win} = \mathrm{Attn}(q_t, k_{t-w+1:t}, v_{t-w+1:t})$. (In GQA, p_t^{slc} is summed over heads in a group so the group selects the *same* blocks — a hardware requirement.) The pooling in *(i)* is *sequence-length* compression, but on KV only — the L queries are untouched, so the branch still emits one vector per position (per the boxed note above).

How the branches combine — the gate. Each branch runs ordinary attention against the *same* query, $o_t^c = \mathrm{Attn}(q_t, K_t^c, V_t^c)$ for $c \in \{\mathrm{cmp}, \mathrm{slc}, \mathrm{win}\}$. A small per-head MLP on the token's hidden state h_t emits one gate per branch, and the three outputs are linearly mixed:

$$g_t^c = \sigma(\mathrm{MLP}_g(h_t))^c \in [0,1], \qquad o_t^* = \sum_{c \in \{\mathrm{cmp},\mathrm{slc},\mathrm{win}\}} g_t^c\, o_t^c.$$

The key detail: σ is a *sigmoid*, **not** a softmax — the three gates are *independent*, need not sum to 1, and are computed *per head* from h_t (not from the attention scores). So the model can open or suppress each branch freely. This separation — a *separate KV per branch* plus an *independent learned gate* — is what stops the network from *shortcutting* onto the easy sliding-window branch and forces gradient into all three paths. (Contrast DSA, next section: *no* gate — a single attention over the top-k selected tokens.)

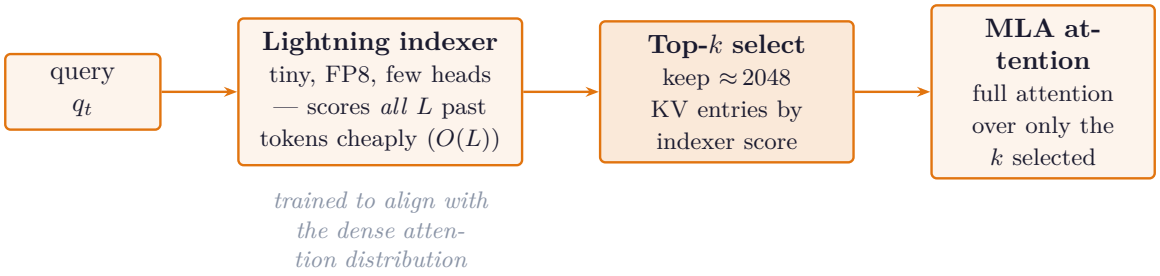
Effect. Trainable sparsity that *matches or beats full attention* while cutting long-context cost, with real wall-clock speedups on both prefill and decode. **Note:** NSA in its paper sits on a conventional (GQA-style) backbone — it attacks the *token-count* axis, *orthogonal* to MLA's per-token axis. DSA is what fuses the two.

3 DSA — DeepSeek Sparse Attention

Lever 1 (inherited MLA) | Lever 3: fine token selection | retrofit, drop-in

Problem. V3.1 is already a strong, *trained* model built on MLA. The goal is NSA-style sparsity as a near **drop-in upgrade**, with minimal continued training rather than pretraining from scratch.

Idea. Two pieces sitting on top of MLA:



The math. A lightweight indexer scores every past token $s \le t$ with a few (H_I) cheap FP8 heads and a ReLU; the expensive MLA attention then runs over the top-k selected tokens *only*:

$$I_{t,s} = \sum_{j=1}^{H_I} w^I_{t,j}\,\mathrm{ReLU}(q^{I\top}_{t,j}\,k^I_{s,j}), \quad \mathcal{S}_t = \text{top-}k(\{I_{t,s}\}_{s\le t}), \quad o_t = \sum_{s\in\mathcal{S}_t} \mathrm{softmax}_s\!\Big(\tfrac{q_t^\top k_s}{\sqrt{d}}\Big) v_s.$$

Scoring is $O(L)$ but tiny ($k \approx 2048 \ll L$); the heavy attention is $O(k)$. The indexer is trained to *align* with dense attention — with $p^{\text{dense}}_{t,\cdot}$ the head-summed, normalised dense distribution, the warm-up loss is $\mathcal{L}_I = \sum_t \mathrm{KL}(p^{\text{dense}}_{t,\cdot} \,\|\, \mathrm{softmax}_s I_{t,s})$.

Training trick. The indexer is first warmed up to imitate the dense attention map (an alignment/KL loss) with the backbone frozen, then the backbone is unfrozen and training continues (∼944B tokens). This makes DSA a *drop-in replacement* for dense attention at near-parity quality.

DSA vs NSA — the precise differences:

- **Granularity:** NSA selects *blocks*; DSA selects *individual tokens* (finer).
- **Selector:** NSA reuses its own compressed-branch scores; DSA adds a *separate* lightweight indexer.
- **Provenance:** NSA is pretrained *from scratch*; DSA is *retrofitted* onto a trained MLA model via continued training.
- **Backbone:** NSA on GQA-style attention; DSA explicitly on *MLA* — so DSA = compression + selection in one layer.

Effect. About **50% lower long-context inference cost** at accuracy parity with the dense (V3.1 "Terminus") baseline — and roughly halved API prices on release.

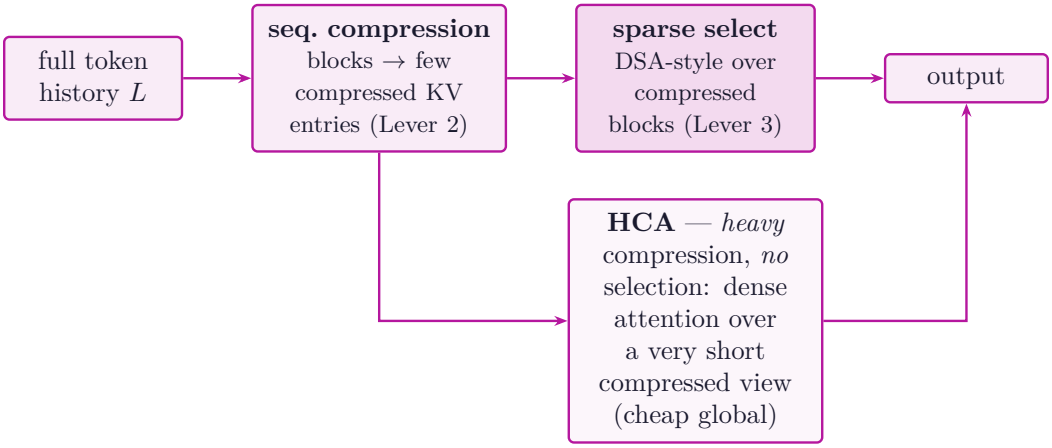
4 CSA (+ HCA) — Compressed Sparse Attention

Lever 1 | Lever 2: sequence compression | Lever 3: selection

Problem. At **1M-token** context, even DSA's $O(L)$ indexing and an $O(L)$ latent cache become expensive. V4 needs to cut both the *number of KV entries* and the *work per query* further.

Idea — compose all three levers. CSA first *compresses the KV cache along the sequence dimension* (group tokens into a smaller number of compressed entries) and *then* runs DSA-style

sparse selection *over the compressed representation*. So selection operates on far fewer, denser entries.



The math. CSA puts a *sequence-dimension* compressor under the selection step. First merge each length-b block of KV into one compressed entry — a *learned* "softmax-gated" pool (a weighted average within each ∼4-token block, $b \approx 4$), *not* a fixed mean — cutting the KV count by b, then run DSA-style top-k' selection *over the compressed entries*:

$$\hat{k}_m = \psi_K\big(k_{(m-1)b+1:\,mb}\big), \;\; \hat{v}_m = \psi_V\big(v_{(m-1)b+1:\,mb}\big), \quad \mathcal{S}_t = \text{top-}k'\big(\{\hat{I}_{t,m}\}_m\big), \;\; o_t^{CSA} = \text{Attn}\big(q_t, \{\hat{k}_m, \hat{v}_m\}_{m\in\mathcal{S}_t}\big).$$

The pool ψ itself is a *learned* weighted average: with a gating vector w,

$$a_{m,i} = \text{softmax}_i\big(w^\top k_{(m-1)b+i}\big), \qquad \hat{k}_m = \sum_{i=1}^{b} a_{m,i}\, k_{(m-1)b+i}, \quad \hat{v}_m = \sum_{i=1}^{b} a_{m,i}\, v_{(m-1)b+i},$$

so plain mean pooling is just the untrained special case $a_{m,i} = 1/b$. (Operator schematic; reconstructed.) HCA uses a much larger ratio $B \gg b$ and *no* selection — the compressed sequence is short enough for dense attention, $o_t^{HCA} = \text{Attn}\big(q_t, \{\hat{k}_m^H, \hat{v}_m^H\}_{m=1}^{\lceil t/B\rceil}\big)$. Both add a *learnable attention sink*: a sink logit $s_\star$ enters the denominator, so the weights may sum to less than one and a query can decline to attend,

$$\alpha_j = \frac{\exp(q_t^\top k_j/\sqrt{d})}{\exp(s_\star) + \sum_{j'}\exp(q_t^\top k_{j'}/\sqrt{d})}.$$

What is compressed (and what is *not*). The pooling is on the *KV cache* along the sequence dimension, not the residual stream. Queries stay full, so the attention map is $L \times M$ ($M \approx L/4$) and the layer still outputs L vectors — there is **no decompression / up-sampling step**, and the pooled $\hat{k}_m, \hat{v}_m$ are attended to *directly* (never reconstructed). One contrast with NSA: NSA's selection branch picks coarse blocks but then re-reads the *original full-resolution tokens* inside them (a real coarse→fine "expand"); CSA attends to the pooled summaries directly — cheaper, lossier, no expansion.

Mental model. Because only K/V shrink while all L queries remain, the map is $L \times M$ — the *same shape as cross-attention*. A compressed layer is really *cross-attention into a memory bank*, with three twists: it is causally masked (the memory is this token's own past), the memory is pooled from the same stream and grows during decoding, and top-k makes it *per-query sparse*. HCA (no selection) is the cleanest case — every query reads the same M pooled entries. (MLA is *not* this: it keeps all L entries, so its map stays square $L \times L$.)

HCA (Heavily Compressed Attention). A companion mode: compress so aggressively that the sequence is short enough for ordinary *dense* attention — no selection needed. It trades selectivity for an ultra-cheap global view. V4 runs a **hybrid of CSA and HCA**. Both add *learnable attention*

sinks (query heads may route some attention mass to a learned sink rather than being forced onto context).

Effect (vs V3.2, at 1M tokens).

	single-token inference FLOPs	KV cache size
DeepSeek-V4-Pro	**27%** of V3.2	**10%** of V3.2
DeepSeek-V4-Flash	**10%** of V3.2	**7%** of V3.2

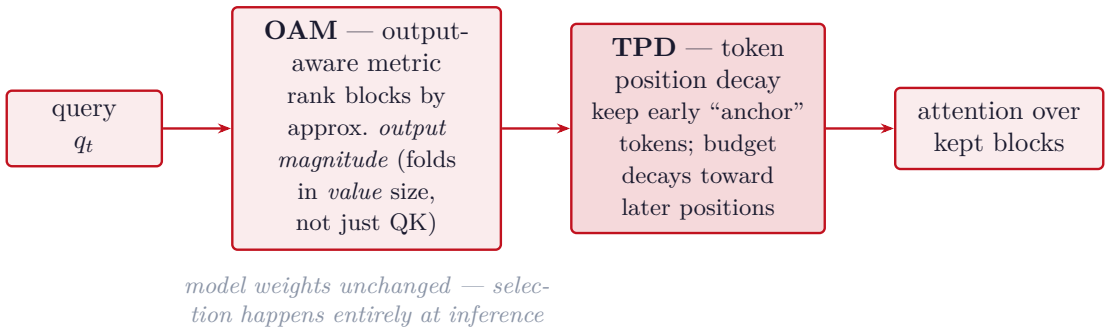
Aside: V4 also replaces residual connections with *Manifold-Constrained Hyper-Connections* (mHC, a doubly-stochastic / Birkhoff-polytope inter-layer mixing for depth stability). That is a separate axis from attention and is out of scope here, but it is why V4 can stack the compressed attention so deep. Concretely the residual is widened to n parallel streams $H_\ell = [h_\ell^1; \ldots; h_\ell^n]$ mixed each layer by a matrix on the Birkhoff polytope (doubly stochastic), $H_{\ell+1} \approx A_\ell H_\ell + \cdots$ with $A_\ell \mathbf{1} = \mathbf{1}$, $A_\ell^\top \mathbf{1} = \mathbf{1}$, $A_\ell \geq 0$ — a norm-preserving mix that keeps deep stacks stable. (Schematic; the exact V4 form is only partially public.)

5 Interlude — Stem, a training-free counterpoint (Tencent)

Lever 3: selection **training-free, plug-and-play** **different lab**

Everything above is one lab's *trained* program: DeepSeek makes the model *learn* its own sparsity (native in NSA, a learned indexer in DSA). Tencent Hunyuan's **Stem** (*Rethinking Causal Information Flow in Sparse Attention*, arXiv:2603.06274, ICML 2026) attacks the same selection lever from the *opposite* direction: keep the model **frozen** and just improve the *rule* that predicts **which blocks of the KV cache each query needs to read**. It is a *plug-and-play* module, not a retraining recipe — the same family as Quest / H2O / MInference, but with a better importance metric.

The two ideas. Prior query-aware selectors rank KV blocks by raw query–key score alone. Stem argues that is the wrong signal and changes two things:



The math. Stem changes the block-importance score itself. Where Quest-style methods rank block j by a query–key proxy alone, the *output-aware metric* estimates each block's contribution *magnitude* (value-aware) and *token position decay* multiplies in a position factor that protects early anchors:

$$s_{t,j}^{OAM} \approx \tilde{\alpha}_{t,j} \, \|\hat{v}_j\|, \qquad \tilde{s}_{t,j} = \underbrace{\phi(\mathrm{pos}_j)}_{\text{TPD, } \phi\downarrow \text{ with position}} \cdot s_{t,j}^{OAM}, \qquad \mathcal{S}_t = \text{top-}n(\tilde{s}_{t,\cdot}),$$

where $\tilde{\alpha}_{t,j}$ is the approximate attention mass on block j, $\hat{v}_j$ a block value summary, and ϕ a decreasing position weight (equivalently, a larger top-n budget for early positions). The model stays *frozen* — only the selected set $\mathcal{S}_t$ changes. (Form schematic, from the paper's description.)

- **Output-Aware Metric (OAM):** a block with a modest attention weight but a *large value vector* still moves the output. Ranking by approximate output magnitude (value-aware) keeps those blocks that pure QK scoring drops.
- **Token Position Decay (TPD):** a *position-dependent* budget — initial tokens behave as recursive anchors and are retained with high stability, while later positions (more redundant) are sparsified harder.

Effect. Dense-level accuracy at **∼25% of the compute**, and **3.7× lower first-token (prefill) latency at 128K** via the open-source block-sparse operator.

Stem vs. the DeepSeek line — the philosophical split:

	DeepSeek NSA / DSA	Tencent Stem
Selector	**trained** (native sparsity / learned indexer aligned to dense attn)	**training-free** heuristic, frozen model
Picks blocks by	learned importance / QK-style indexer score	*output magnitude* (OAM, value-aware) + position budget (TPD)
Integration	baked into pre- / continued-training	drop-in at inference (plug-and-play)
Bet	let the model *learn* what to keep	a smarter *rule* beats raw QK ranking

In the three-lever picture Stem pulls only Lever 3 — but it is the cleanest demonstration that the *selection metric itself* (value-awareness, position structure) is an independent axis of progress, separate from whether the sparsity is trained in.

6 Aside — how is a non-differentiable top-k trained?

applies to NSA, DSA, CSA

Selection — NSA's block pick, DSA's and CSA's token/entry pick — all rest on top-k, a *hard, discrete* choice with **no gradient**: you cannot backpropagate "token 5 was kept, token 6 was not." None of these methods try to. The fix is always the same idea:

never differentiate the pick; differentiate the work done on the picked tokens, and train the scorer some other way.

1. Gradient still reaches the model normally. Once the set $\mathcal{S}_t$ is chosen, the attention $o_t = \mathrm{Attn}(q_t, \{k_s, v_s\}_{s\in\mathcal{S}_t})$ is an *ordinary* differentiable softmax. The selected tokens' k_s, v_s and the query q_t get gradients as usual; unselected tokens simply get none that step. The selection mask is treated as a constant (a stop-gradient, $\mathrm{sg}[\,\mathcal{S}_t\,]$) — exactly like hard routing in a Mixture-of-Experts.

2. The scorer is trained *without* passing through top-k. This is the only method-specific part:

- **DSA / CSA (the indexer):** train the indexer with a *separate, fully differentiable* target — the alignment loss $\mathcal{L}_I = \mathrm{KL}(p^{\mathrm{dense}} \,\|\, \mathrm{softmax}(I))$ that teaches it to score tokens the way real *dense* attention would. The indexer learns "what to pick" from this soft supervision; the hard top-k is used only on the forward pass. (During warm-up the backbone is frozen, so this loss is the *only* thing the indexer learns from.)
- **NSA:** no extra loss at all — the importance scores *are* the compression branch's own attention probabilities p^{cmp}. That branch is a real, gated, differentiable part of the output, so its scores are trained to be meaningful through the normal end-to-end loss; selection just *reuses* them. (The arg top-n itself is still stop-gradient.)

- **Stem:** nothing to train — the model is frozen, so top-k is a pure inference-time choice and non-differentiability never arises.

So top-k is **forward-only routing**. Learning *to select* is offloaded either to a differentiable proxy loss (DSA/CSA) or to a parallel differentiable branch whose scores are reused (NSA) — never to the discrete operator itself. It is the same trick as hard attention and sparse MoE: don't differentiate the decision, differentiate the computation around it and supervise the router separately.

7 Side-by-side

	MLA	NSA	DSA	CSA / HCA
First in	DeepSeek-V2	native (ACL'25)	DeepSeek-V3.2	DeepSeek-V4
Attacks	per-token KV size	token count	token count	both + seq. length
Levers	1	3 (+ coarse 2)	1 + 3	1 + 2 + 3
Tokens seen	all (dense)	top-n blocks	top-k tokens	selected compressed blocks
Selector	— none —	compressed-branch scores	separate lightning indexer	indexer over compressed entries
Granularity	—	block	token	compressed block
Trained	from scratch	from scratch (native)	retrofit / continued	from scratch
Backbone	introduces MLA	GQA-style	on top of MLA	MLA + seq. compress
Headline win	∼14× smaller KV	full-attn quality, sparse cost	∼50% long-ctx cost	1M ctx: 7–10% KV

8 Reading the lineage

The four methods are not competitors; they are **accumulating levers on the same machine**.

- **MLA** made each token *cheap to remember* (compression), but still reads everything.
- **NSA** proved you can *read fewer tokens* without quality loss — if sparsity is native and block-structured for the GPU.
- **DSA** *fused* those two ideas and, crucially, showed the fusion can be *retrofitted* onto an existing MLA model with a cheap learned indexer rather than a from-scratch redesign.
- **CSA/HCA** added the third lever — *compress the sequence itself* before selecting — to push into the million-token regime, and split it into a precise mode (CSA) and an ultra-cheap global mode (HCA).

The consistent design instinct: *move work off the critical path and make whatever remains hardware-shaped.* Compression shrinks what must be stored; native, block/entry-structured selection shrinks what must be computed; and each generation keeps the part that was expensive (the indexer, the

compression) *small and trainable* so the model learns its own sparsity instead of having it imposed. Each step is also a small *deletion* — of cache, of attended tokens, of sequence length — which is the recurring DeepSeek signature.

Note on sources: MLA, NSA, and DSA mechanisms are from the DeepSeek-V2, NSA (arXiv:2502.11089), and DeepSeek-V3.2 (arXiv:2512.02556) papers. V4 / CSA / HCA figures are from DeepSeek's V4 model card and accompanying technical write-ups (Apr 2026); some V4 internals are described at a higher level than the earlier papers, so the CSA/HCA mechanism is reported as published rather than re-derived. Stem is from Tencent Hunyuan (arXiv:2603.06274, ICML 2026); it is included as an external, training-free contrast to the DeepSeek line rather than part of it. Equations for MLA, NSA and DSA follow the respective papers; those for CSA/HCA, mHC and Stem are schematic reconstructions of the described mechanisms, whose exact forms are only partially public.